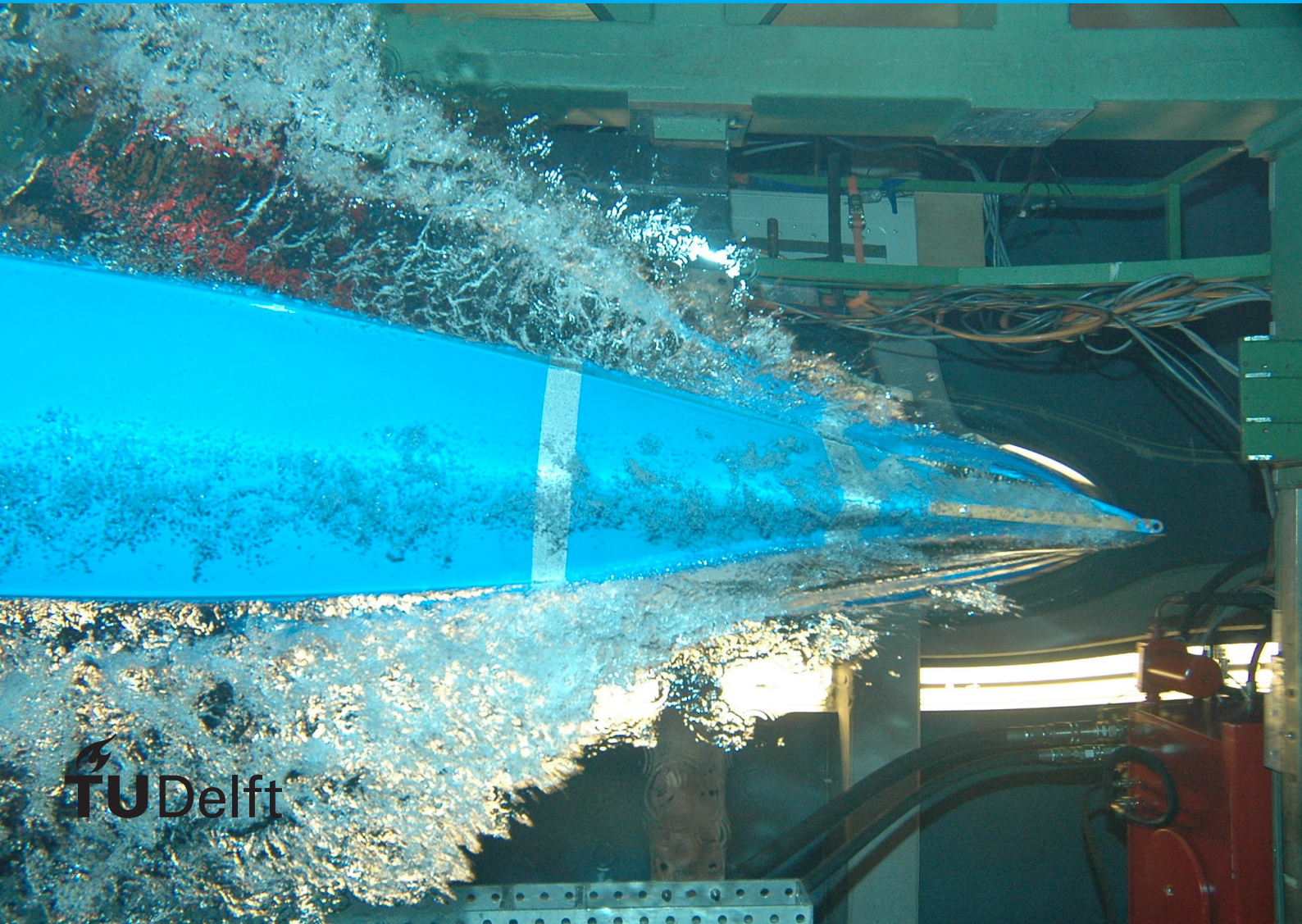
Policy Gradient Reinforcement Learning for quadrotor control

Preliminary thesis

A. M. de Windt



Policy Gradient Reinforcement Learning for quadrotor control

Preliminary thesis

by

A. M. de Windt

to obtain the degree of Master of Science

at the Delft University of Technology,

to be defended publicly on Tuesday January 1, 2013 at 10:00 AM.

Student number:	1234567	
Project duration:	March 1, 2012 – January 1, 2013	
Thesis committee:	Prof. dr. ir. J. Doe,	TU Delft, supervisor
	Dr. E. L. Brown,	TU Delft
	Ir. A. Aaronson,	Acme Corporation

This thesis is confidential and cannot be made public until December 31, 2013.

An electronic version of this thesis is available at `http://repository.tudelft.nl/`.

TU Delft

Todo list

Rewrite this paragraph so it's easier to read. 1
Expand a bit more here. 1
Add examples of actual use cases and research instead of a simple, ranging from ... to 1
Rewite this paragraph. 4
Rewrite this paragraph. 6
Add reference to the transition probability function explanation in previous chapter. 7

Preface

Preface…

A. M. de Windt
Delft, January 2013

List of Figures

2.1 Example reward distribution for a 5-armed bandit problem. 5
2.2 Average reward of 2000 10-armed bandit problems for 1000 episodes when following the greedy and two ϵ-greedy policies. 6

List of Tables

Acronyms

DP Dynamic Programming.

MDP Markov Decision Process.

RL Reinforcement Learning.

Symbols

G_t return (cumulative discounted reward) following time t.

$G_t^{\lambda a}$ λ-return, corrected by estimated action values.

$G_t^{\lambda s}$ λ-return, corrected by estimated state values.

$G_{t:h}$ flat return (uncorrected, undiscounted) from $t+1$ to h.

$G_{t:h}^{\lambda a}$ truncated, corrected λ-return, with action values.

$G_{t:h}^{\lambda s}$ truncated, corrected λ-return, with state values.

Q, Q_t array estimates of action-value function q_π or q_*.

V, V_t array estimates of state-value function v_π or v_*.

$\in$ is an element of, e.g., $s \in \mathcal{S}$.

$|\mathcal{S}|$ number of elements in $\mathcal{S}$.

$\mathcal{A}$ set of all actions.

$\mathcal{R}$ set of all possible rewards, a finite subset of $\mathbb{R}$.

$\mathcal{S}$ set of all states.

$\mathcal{S}^+$ set of all states, including the terminal state.

$\subset$ subset, e.g., $\mathcal{R} \subset \mathbb{R}$.

a an action.

$p\left(s', r | s, a\right)$ probability of transition to state s' with reward r, from state s and action a.

$p\left(s' | s, a\right)$ probability of transition to state s', from state s and action a.

$q_*(s, a)$ value taking action a in state s under the optimal policy.

$q_\pi(s, a)$ value taking action a in state s under policy π.

r a reward.

$r\left(s, a, s'\right)$ expected immediate reward on transition from s to s' under action a.

s a state.

$v_*(s)$ value of state s under optimal policy.

$v_\pi(s)$ value of state s under the optimal policy π (expected return).

Greek symbols

π policy, decision-making rule.

$\pi(a|s, \boldsymbol{\theta})$ probability of taking action a in state s given parameter $\boldsymbol{\theta}$.

$\pi(a|s)$ probablity of taking action a in state s under *stochastic* policy π.

$\pi(s)$ action taken in state s under *deterministic* policy π.

Contents

List of Figures vii

List of Tables ix

1 Introduction 1

2 Introduction into Reinforcement learning 3

2.1 Finite Markov Decision Process . . . 3
2.2 Elements of Reinforcement Learning . . . 3
2.3 Expected return . . . 4
2.4 Policy and value functions . . . 5
2.5 K-armed bandit problem . . . 5
2.6 ϵ-greedy policies . . . 6

3 Tabular Methods 7

3.1 Dynamic Programming . . . 7
3.1.1 Policy evaluation . . . 7
3.1.2 Policy improvement . . . 8

4 Approximate solutions 11

5 Past, present and future of Reinforcement Learning 13

6 Conclusion 15

7 Proposed thesis research 17

1 Introduction

Rewrite this paragr it's easier to read.

The technological advancements and the steadily increasing demand for air and space travel of the past decades have lead to the development of new, more complex aerospace systems, resulting in new challenges in the field of aerospace control. At the same time, advancements in automated control have been seen in many domains, ranging from industrial manufacturing and household appliances to the rapid widespread adoption of UAVs. However, most of these systems are only able to complete their tasks within the narrow, predictable conditions they where designed for. Autonomous control systems need to be able to adapt to unexpected situations and environments. This has motivated researchers to search for new innovative methods that can be used to solve these new problems.

Expand a bit more

One of the leading causes of fatal air traffic accidents is loss of control. If fully autonomous UAV's are to be used in the future, these need to be at least as safe as a human piloted aircraft. However since it is impossible to take every known and unknown failure case into account when designing flight control systems, it is necessary to create fault tolerant control systems capable of identifying and adapting to changing conditions.

Furthermore, thanks to the miniaturization of technologies, Micro Air Vehicles are becoming smaller, more diverse and are seeing a rise is use. However, these types of vehicles come with their own new problems. In order to design a reliable, well performing control system, an accurate dynamics model is needed. This is not always possible for MAVs because of the complex nature of MAV flight dynamics, inconsistent manufacturing and parameter adaptation during time. Because of their smaller size, MAVs are more susceptible to unforseen disturbances and have sensor and actuator limitations due to their weight and size requirements.

Adaptive control methods allow autonomous systems to reconfigure them self if necessary. This opens the possibility to create fault tolerant control systems and control systems capable of determining the governing dynamics and track changes that may occur over time due to degradation or varying flight conditions. These systems can also detect and compensate for the limited on-board resources and any inconsistencies that may have occurred during manufacturing.

Add examples of a cases and research of a simple, rangin ... to ...

Reinforcement learning is a framework of intelligent, self-learning methods that can be applied to a wide range of autonomous operations and applications. These methods have been used in robotics for decades and a lot of research has been taking place at the same time in a wide range of topics, from resource management in computer clusters to optimizing chemical reactions. However, most of these use cases involve taking high level decisions such as trajectories, schedules, etc, and not low level actuator control.

RL methods are also relatively new in the field of aerospace and it's thus uncertain which systems can make effective use of the advantages of RL. This thesis proposes the development of online, self tuning low level quadcopters controllers.

The objective of this thesis is to develop a low level quadrotor controller with Reinforcement Learning elements and investigate it's effectiveness compared to existing methods.

What is Reinforcement Learning and what can it be used for?
How does Reinforcement Learning learning work?
What is Reinforcement Learning capable of?

How has Reinforcement Learning been used so far?
What state of the art technological developments make use of Reinforcement Learning?

How can Reinforcement Learning methods be used to improve the performance of low level control of a quadcopter?
What advantages do RL based controllers provide over classical control methods?
What RL methods can be effective when used as part of the low level control of a quadrotor?
What controller structures can make effective use of RL?
How do these new methods compare to classical control methods?
Are these RL based controllers fault tolerant and capable of adapting themselves in a practical environment?

2

Introduction into Reinforcement learning

Reinforcement Learning (RL) is a method of machine learning different from other machine learning methods such as supervised learning which uses a set of labeled examples from a knowledgeable supervisor, and unsupervised learning which is typically used to find hidden patterns in unlabeled data. In Reinforcement Learning (RL) the learner is not told what action to take to achieve its goal, but instead it maps situations and actions by trying them out and discovering the most valuable. It does this by not only looking at the reward it received by its latest action, but it also takes into account any future rewards it could receive in the future by taking that action.

2.1. Finite Markov Decision Process

A Markov Decision Process (MDP) is a stochastic process that provides a mathematical framework used to describe problems where a learner learns from its actions to achieve an end goal. This framework is used in many areas, such as robotics, economics, manufacturing and in the case of this report, Reinforcement Learning.

The two main components of a Markov Decision Process (MDP) are the agent, which makes decisions, interacts with its environment and learns. And the environment which consists of everything else. The two are constantly interacting with each other, the agent will perform an action which changes the state of the environment and the environment responds by giving it a reward. The goal of the agent is to maximize the reward it receives from the environment.

This happens in a sequence of discrete time steps. At the beginning of each time step the environment is at a certain state s. The agent then decides based on its policy π which action a to take, this in turn changes the environment to the new state s' and the environment then gives the agent a certain reward r. In order for it to be a MDP this transition and the reward must satisfy the Marcov property, which means it needs to be memoryless and thus the environment's behavior must be independent of its past states. In other words, the probability of the environment to go from state s to s' and the reward r it gives back to the agent, must only be dependent on the initial state s and action a.

2.2. Elements of Reinforcement Learning

The main goal when solving an MDP problem is to find a policy π that defines the action $\pi(s)$ the agent will take in order to maximize the sum of rewards it receives. It characterizes the way in which the agent will behave at a given time by mapping the agents perceived state of the environment to the actions taken in those states. This policy may be as simple as a table or consist of several Neural Networks. The policy is the core of the agent in that it alone defines the behavior of the agent.

On each time step the environment will send the agent a reward. The goal of the agent is to maximize this reward in the long run. Thus this reward is used to tell the agent which states are good or bad. If the agent receives a low reward for a certain action it can change the policy so it takes a different action in the future.

The reward signal tells the agent what the immediate effects of its action are. However, its goal is to maximize

the cumulative sum of rewards it receives over time, and thus in order for it to receive higher rewards in the future, it might be necessary for the agent to take actions that yield lower rewards with the promise that it will receive higher rewards later. So the value is an indication of the reward the agent can expect to receive in the future by taking a certain action, and thus it is the main parameter used in the decision making process.

By defining the state and action taken during step t as S_t and A_t respectively, and its corresponding reward as R_t we can define the true value of an arbitrary action a as:

$$q_*(s,a) \doteq \mathbb{E}[R_t \mid S_t = s, A_t = a] \tag{2.1}$$

If $q_*(s,a)$ was known, then the solution would be trivial. Just select the action with the highest value. So it's assumed that the action values are not known, but are instead estimated from the rewards the agent receives. The estimated value of the state-action pair (s,a) at time t is known as $Q_t(s,a)$, thus $Q_t(s,a)$ needs to be as close to $q_*(s,a)$ as possible.

Finally some RL agents make use of a model for planning. This allows these agents to choose actions based on possible future situations before actually being experienced.

2.3. Expected return

The goal of the agent is to maximize the reward it receives over time. In order to formalize this goal, the *expected return* G_t can be defined as a function of the sequence of rewards. In it's simplest form this can be the cumulative sum of all rewards after time step t until the final time step T.

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \ldots + R_T \tag{2.2}$$

This function assumes that there are one or more terminal states where the agent-environment interaction ends and break naturally into subsequences called *episodes*. A good example are plays of a game or any type of repeating task. Even if there are different terminal states, eg. winning or losing a game, the starting state of the next episode must be independent of the previous episode. These type of tasks are called *episodic tasks*.

However not all tasks are episodic, it is possible an RL-agent needs to solve some task that can continuo without a limit. These types of tasks are called *continuing tasks*. The expected return given by Eq. 2.2 can be problematic for these types of tasks, since the final time step would be at $T = \infty$. Thus it is possible the value it's trying to maximize is infinite as well. Thus to solve this problem the rewards can be $discounted$. In this approach the sum of discounted rewards, also known as the *discounted expected return*, shown in Eq. 2.3 is maximized.

is paragraph.

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.3}$$

It is possible to unify the expected return equations for the episodic and continuing tasks into a single one. For example, in the transition diagram for a continuos task shown below the environment transitions from states s_0 to s_1 and after its next action it ends up at a special absorbing state where all actions will result in the same state without any reward. This results in reward sequence $+1, +1, +1, 0, 0, \ldots$. By summing the whole infinite sequence we get the same result as summing the first T rewards, where $T = 3$ in this case. This is also true with discounting.

Thus Eq. 2.4 shows the unified equation for the (discounted) expected return for both episodic and continuing tasks. If the task at hand is continuing, then $T = \infty$, while for episodic tasks T will be some finite integer. It is possible to have $T = \infty$ for episodic tasks, but due to the reasons mentioned before the expected return must be discounted for continuing tasks, so $\gamma = 1$ and $T = \infty$ cannot true at the same time.

$$G_t \doteq \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \tag{2.4}$$

One important property of the expected return used in reinforcement learning is that it is recursive as demonstrated below.

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma \left(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots \right) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned} \tag{2.5}$$

2.4. Policy and value functions

The policy is a mapping from the state of the environment to the probabilities of the agent selecting each action. If the policy at time t is π, then $\pi(a|s)$ is the probability of selecting action a if the current state is s.

The value of a state can be different depending on the policy of the RL-agent, since different policies will result in different actions and thus different states and rewards. So $v_\pi(s)$ is the expected return when starting at state s and following policy π. v_π can be defined by:

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t | S_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \middle| S_t = s \right], \text{ for all } s \text{ in } \mathcal{S} \tag{2.6}$$

For some problems it might be more suitable to look at the value of state-action pairs. Thus similar as to the state value above, the state-action value under policy π, denoted by $q_\pi(s,a)$, is defined by:

$$q_\pi(s,a) = \mathbb{E}_\pi \left[G_t | S_t = s, A_t = a \right] = \mathbb{E}_\pi \left[\sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \middle| S_t = s, A_t = a \right], \text{ for all } s \text{ in } \mathcal{S} \tag{2.7}$$

2.5. K-armed bandit problem

K-armed bandit problems will be used as examples in the following sections. These are simple learning problems where the agent is given the option to take k different action's. There is only one state and the episodes terminate after the agent has taken it's action. Furthermore the examples in this report use zero mean, unit variance normal distributions to select the true value $q_*(a)$ of the actions. The rewards given by the environment are then selected with $q_*(a)$ mean and unit variance normal distribution. Figure 2.1 shows an example reward distribution for a 5-armed bandit.

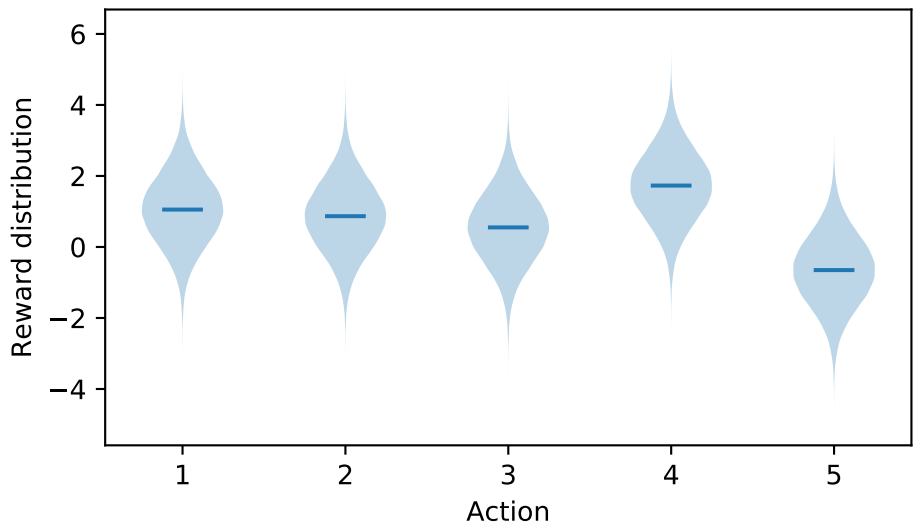


Figure 2.1: Example reward distribution for a 5-armed bandit problem.

2.6. ϵ-greedy policies

The main difference between Reinforcement Learning and other types of machine learning is that RL uses training information that evaluates the actions taken by the agent instead of a set of correct actions. This means that in order to determine $Q_t(a)$ the agent needs to explore and try out as many actions as possible in order to find good behavior. To do this the agent can take random actions and learn from the rewards it gets back. This is called *exploration*.

However, the main goal of the agent is to maximize the rewards it receives from the environment over time. To achieve this goal the agent can choose to take the greedy actions. These are the actions with the highest value in $Q_t(a)$. This process of choosing the greedy action is known as *exploitation* since the agent is exploiting its knowledge to achieve its goal.

Unfortunately exploration and exploitation conflict with each other since in the former the agent wants to try out all possible actions, but only wants to perform a specific action if it's following a greedy policy. Thus a possible compromise is to use what is known as an ϵ-greedy policy. In these policies, the agent will choose a random action ϵ percentage of the time and the greedy action the rest of the time. This allows the agent to explore and try out new actions or old actions whose value might have changed, while also trying to achieve it's goal.

his paragraph.

Since the agent is trying out new actions while following ϵ-greedy policies, it is able to learn faster and more than if it was following a greedy policy. To demonstrate this 2000 10-armed bandit problems are run for 1000 episodes. The average reward of all bandits are stored after each episode. Figure 2.2 shows the average reward and the percentage of agents that selected the optimal action for the greedy policy and two ϵ-greedy policies ($\epsilon = 0.1$ and $\epsilon = 0.1$). A full explanation of the learning method used can be found in the section ????, but in short, the action value function $Q_t(a)$ is estimated by averaging the rewards received from the environment.

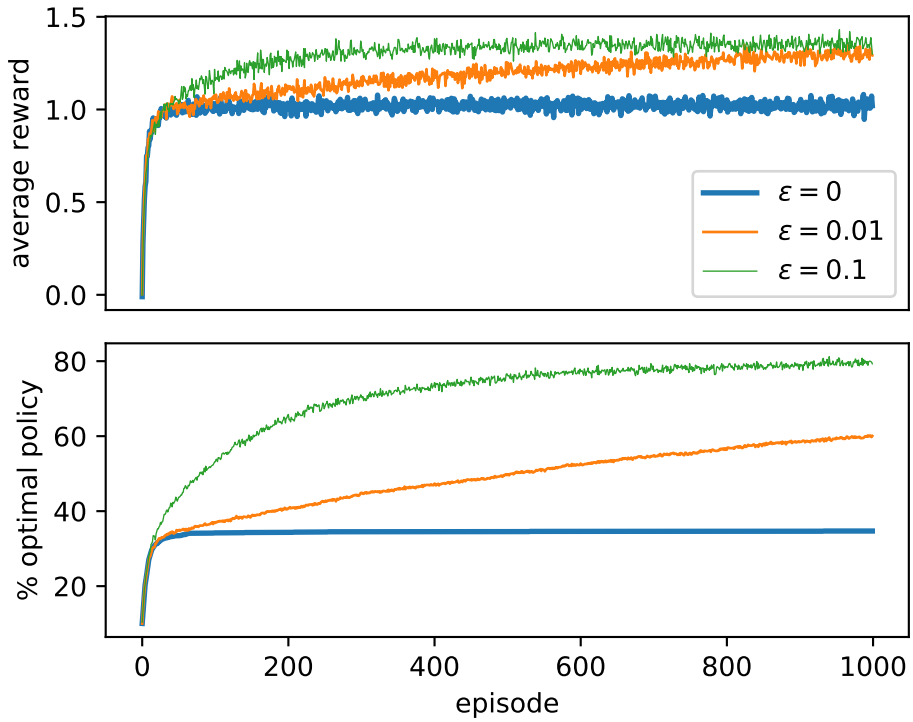


Figure 2.2: Average reward of 2000 10-armed bandit problems for 1000 episodes when following the greedy and two ϵ-greedy policies.

As it can be seen when using ϵ-greedy policies the agents learn faster and are thus able to receive higher rewards sooner. On the other hand, while following the greedy policy, the agents will quickly choose the first action they find with a reward higher than the initial action value and then perform that action on all episodes. As a results many of the agents are not able to find the optimal policy.

3

Tabular Methods

The action-value function estimate $Q_t(s,a)$ holds the current estimate of the real action-value function $q_*(a)$. If the RL-agent is to achieve it's goal, $Q_t(s,a)$ needs to be as close to $q_*(a)$ as possible. The actual function used to represent $Q_t(s,a)$ can be different depending on the application, and the methods used to estimate $Q_t(s,a)$ can be different depending on the underlying function and application.

This chapter will focus on methods used to solve Markov Decision Process (MDP) problems where the value and policy consist of tables of discreet values. These methods are useful with small discreet state- and action-spaces and can be used to explain different concepts. The chapter will first start with Dynamic Programming and Monte Carlo methods. These methods have limited use in Reinforcement Learning (RL), but provide an essential base for understanding the inner workings of RL methods.

3.1. Dynamic Programming

Dynamic Programming (DP) refers to a collection of methods used to solve MDP problems. The main idea of DP and RL is the use of value functions to organize and structure the search for good policies. The optimal policies can be easily be found once the optimal value functions v_* or q_* which satisfy the Bellman optimality equations shown below are known.

Add reference to th
sition probability f
explanation in prev
chapter.

$$\begin{aligned} v_*(s) &= \max_a \mathbb{E}\left[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_*(s')\right] \end{aligned} \tag{3.1}$$

$$\begin{aligned} q_*(s,a) &= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1},a') | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s',r|s,a)\left[r + \gamma \max_{a'} q_*(S_{t+1},a')\right] \end{aligned} \tag{3.2}$$

DP algorithms are found by turning the Bellman equations into assignment and, or update rules for improving approximations of the value functions.

3.1.1. Policy evaluation

The first step in DP is to compute the *state-value* function v_π for an arbitrary policy π. This is called *policy evaluation* or *prediction problem*.

This can be done by writing the state-value function v_π from eq. 2.6 in terms of the policy probability function $\pi(a|s)$ and the transition probability function $p(s',t|s,a)$. The policy probability function is the probability of action a being selected by the policy while being in state s. And the transition probability function gives the probability of the environment transitioning to state s' and giving reward r if action a is taken while in state s.

$$
\begin{aligned}
v_\pi(s) &\doteq \mathbb{E}_\pi\left[G_t|S_t = s\right] \\
&= \mathbb{E}_\pi\left[R_{t+1} + \gamma G_{t+1}|S_t = s\right] \\
&= \mathbb{E}_\pi\left[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s\right] \\
&= \sum_a \pi(a|s) \sum_{s',r} p(s',t|s,a)\left[r + \gamma v_\pi(s')\right]
\end{aligned} \tag{3.3}
$$

If the environment's dynamics are known, then Eq. 3.3 is a system of $|\mathcal{S}|$ equations with $|\mathcal{S}|$ unknowns. Solving it would be tedious, but trivial. However, this is almost never the case, thus iterative solution methods provide a better approach. Consider a sequence of value functions $v_1, v_2, v_3, \ldots$, each mapping $\mathcal{S}^+$ to $\mathbb{R}$. The initial value function approximation may be chosen arbitrarily and new approximations are calculated using the Bellman equation for v_π as an update rule.

$$
\begin{aligned}
v_{k+1}(s) &= \mathbb{E}\left[R_{t+1} + \gamma v_k(S_{t+1})|S_t = s, A_t = a\right] \\
&= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_k(s')\right]
\end{aligned} \tag{3.4}
$$

The state-value function v_π is a fixed point for this update rule because the Bellman equation for v_π assures us of equality in this case. It is also possible to show that the sequence v_k approaches v_π as $k \to \infty$. This algorithm is called the *iterative policy evaluation.*

To produce each approximation the iterative policy evaluation applies the operation to each state s and replacing the old value of s with a new one based on the immediate received reward and the old values of the successor states of state s. This operation is called an *expected update.*

One way to implement this algorithm is to keep two arrays, one to hold the old values and another for the new ones. This way the new values can be computed from the old ones, without changing the old ones. It is also possible to implement it with one array and update the values "in place", by immediately replacing the old value by the one. The in-place algorithm not only consumes less memory, but also usually converges faster than the two-array version. This is because the new values are used as soon as they are available.

3.1.2. Policy improvement

With the value function $v_\pi(s)$ for a deterministic policy π known, it is possible to check for each state, whether there is an action a which yields a larger value than the action taken by the current policy π. One way to do this, is for each state s, select an action a different from the action taken by the current policy ($a \neq \pi(s)$) and calculate the value if that action is taken first, followed by the current policy after that. The value for this behaviour is:

$$
\begin{aligned}
q_\pi(s,a) &= \mathbb{E}\left[R_{t+1} + \gamma v_\pi\left(S_{t+1}\right)|S_t = s, A_t = a\right] \\
&= \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_\pi\left(s'\right)\right]
\end{aligned} \tag{3.5}
$$

If this value is greater than that of the current policy, then a better action has been found. It can therefore be expected that a policy where the new action a is taken whenever state s is encountered is a better policy than the current policy. Thus updating the policy π with the new action will result in a new policy that is as good as or better than the current policy.

Repeating this process for all state-action combinations, it is possible to find a new greedy policy π', given below. Where $\arg\max_a$ denotes the value of a which maximizes the expression that follows.

$$
\begin{aligned}
q_\pi(s,a) &\doteq \arg\max_a \; q_\pi(s,a) \\
&= \arg\max_a \; \mathbb{E}\left[R_{t+1} + \gamma v_\pi\left(S_{t+1}\right)|S_t = s, A_t = a\right] \\
&= \arg\max_a \; \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_\pi\left(s'\right)\right]
\end{aligned} \tag{3.6}
$$

The process of making a new policy that improves over the original one using the values of the original policy is called *policy improvement*. In the case where the new policy π' is as good than the old policy π, Then $v_{\pi'} = v_\pi$ and from there is follows that for all $s \in \mathcal{S}$:

$$\begin{aligned} v_{\pi'}(s) &= \max_a \mathbb{E}\left[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s', r|s, a)\left[r + \gamma v_*(s')\right] \end{aligned} \tag{3.7}$$

4

Approximate solutions

5

Past, present and future of Reinforcement Learning

6

Conclusion

7

Proposed thesis research